

STEPS FOR SIMAR'S AI

15 September 2026. Simar works with an AI on the "stock tracker main" Google Sheet and its Apps Script. This is the work, written so it can be pasted into that AI with the product list (`products.csv`) attached. Four tasks, each with the exact file we take back. Our import checks every row and refuses a file with a problem, naming the line, so a file that passes here loads first time.

THE PROMPT TO PASTE

You are helping Nitya Stones' warehouse (a Google Sheet called "stock tracker main" with tabs for Products, Inventory and Stock Transactions, and an Apps Script) hand its data to a new stock system. The new system's product list is attached as `products.csv`: one row per product line, with a ``code`` column, and for mixed patio packs one row per slab size sharing the same code (component_size is filled). Match rows by material, colour, size and thickness as they appear in the sheet; ignore case, spaces and underscores in names.

Do the four tasks below in order. For each, produce exactly the file described, UTF-8 CSV with a header row, and list anything you could not match rather than guessing. Never invent a quantity, a date or a code.

TASK I: PUT THE CODES ON THE SHEET

Add a column `code` to the Products tab (and to Inventory and Stock Transactions if they reference products by name), filled by matching each row to `products.csv` on material, colour, size and thickness.

- Matching is case-insensitive; `_` and extra spaces are ignored; `600/900` and `600x900` and `900 × 600` are the same size.
- A mixed patio pack row (mode `mixed`) matches on material, colour and thickness; all four size rows get the same code.
- Anything that does not match goes in `unmatched.csv`: `sheet_row, material, colour, size, thickness, why`.

An Apps Script shape for it, to adapt to the real column letters:

```
function addCodes() {
  const ss = SpreadsheetApp.getActive();
  const codes = ss.getSheetByName('codes').getDataRange().getValues(); // products.csv pasted
  into a tab named codes
  const key = r => String(r.join('|')).toLowerCase().replace(/[_\s]+/g, '').replace(/x|x/g,
  '/');
  const map = new Map(codes.slice(1).map(r => [key([r[2], r[3], r[4], r[5]]), r[0]])); //
  material, colour, size, thickness -> code
  const sh = ss.getSheetByName('Products'), data = sh.getDataRange().getValues();
  const col = data[0].indexOf('code') + 1 || data[0].length + 1; sh.getRange(1,
  col).setValue('code');
  data.slice(1).forEach((r, i) => { const c = map.get(key([r[0], r[1], r[2], r[3]])); if (c)
  sh.getRange(i + 2, col).setValue(c); });
}
```

TASK 2: DECIDE THE FLAGGED ROWS

`products.csv` has a `note` on eleven rows: a colour listed twice at the same size and thickness, and four porcelains named differently from the store. Produce `decisions.csv`: `code`, `sheet_row_to_keep`, `correct_name`, `comment`. Where two rows share a code, say which one is current and whether the other should be retired. Where a name differs from the store's, confirm or correct the store name we have matched: Clorado Light Summer = Beige, Clorado Dusk Grey = Light Grey, Clorado Light Midnight = Black, HS Biege = HS Beige.

TASK 3: THE OPENING BALANCE

From the Inventory tab, produce `opening_balance.csv` with the quantity on the ground for every active code, counted or confirmed on one day:

```
code,quantity,unit,bay,counted_on,counted_by,note
SS-RAJGREE-MIX-22,238,slab,yard,2026-09-22,simar,4 packs less 2 loose
PC-QUARWHIT-600X900-20,120,slab,yard,2026-09-22,simar,3 pallets
CH-JOINBLAC-15KG,11,bucket,office,2026-09-22,simar,
```

- `unit` is the product's base unit from `products.csv` (`slab`, `box`, `bucket`, `piece`), never pallets: a pallet count is multiplied by `per_pallet`, a mixed pack by the pack's slab count (the sum of its size rows).
- One row per code (a mixed pack is one row, its slabs added together). `bay` is `yard` unless the sheet knows better. `counted_on` is the date of the count, as YYYY-MM-DD.
- Zero rows may be left out. Negative numbers are refused.

TASK 4: THE HISTORY (OPTIONAL)

From the Stock Transactions tab, produce `transactions.csv`:

```
at,code,type,quantity,unit,bay,reference,user,note
2026-08-01 09:15:00,PC-QUARWHIT-600X900-20,goods_in,80,slab,yard,DN 4471,simar,
2026-08-03 11:40:00,PC-QUARWHIT-600X900-20,sale,49,slab,yard,INV 1023,office,1 pallet + 9
slabs
2026-08-05 16:00:00,PC-QUARWHIT-600X900-20,breakage,2,slab,yard,,warehouse,forklift
```

- `type` is one of `goods_in`, `sale`, `dispatch`, `collection`, `breakage`, `adjustment`, `transfer`, `return`. Map the sheet's own transaction types to these and list the mapping in `type_map.csv` (`sheet_type`, `our_type`). Quantities are positive; the type says which way stock moves (an `adjustment` may be negative).
- `at` is the transaction's date and time, `YYYY-MM-DD HH:MM:SS`. Units as in Task 3.
- Oldest first. Rows whose product cannot be matched go in `unmatched_transactions.csv` with the original row.

WHAT HAPPENS ON OUR SIDE

`node scripts/import.mjs opening opening_balance.csv` and `node scripts/import.mjs history transactions.csv` check every row (code exists, quantity is a number, unit matches, type is known, date present) and write the movements into the ledger. A file with a problem loads nothing and returns the list of lines to fix.

Send the files by email or WhatsApp to Fee; there is nothing secret in them.